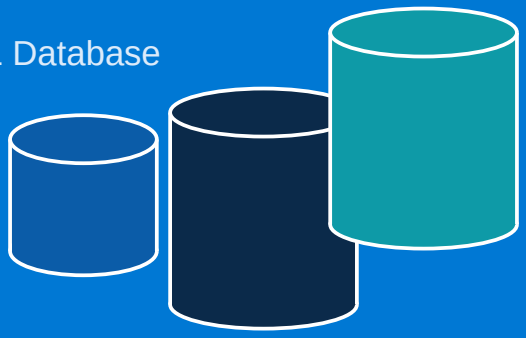
ARCHITECTURE | SECURITY | OPERATIONS | MIGRATION

Azure SQL Architecture Guide

A practical, easy-to-follow handbook for Azure SQL Database and Azure SQL Managed Instance



WHAT YOU WILL LEARN

- Azure basics and the Azure SQL family
- Azure SQL Database: architecture and tiers
- Managed Instance: architecture, networking, tiers
- Security, backup and restore, monitoring
- High availability, disaster recovery and migration

Prepared October 2026. Service limits, tiers and tools change often; verify exact values on Microsoft Learn.

Contents

How to Use This Guide . . . 4

1. Azure Basics . . . 5

1.1 What is Azure and what is 'cloud'? . . . 5

1.2 The shared responsibility model . . . 5

1.3 Azure global infrastructure . . . 6

1.4 How Azure organises resources . . . 6

1.5 Identity: Microsoft Entra ID . . . 7

1.6 Building blocks you will meet in Azure SQL . . . 8

1.7 Ways to work with Azure . . . 8

2. The Azure SQL Family . . . 9

2.1 Meet the three members . . . 9

2.2 Side-by-side comparison . . . 10

2.3 How to choose . . . 10

3. Azure SQL Database . . . 11

3.1 The logical server and databases . . . 11

3.2 Deployment options . . . 11

3.3 Purchasing models: DTU vs vCore . . . 11

3.4 vCore service tiers in depth . . . 12

3.5 Provisioned vs serverless compute . . . 14

3.6 Elastic pools - a worked example . . . 14

3.7 Connectivity architecture . . . 14

3.8 Quick start: create a server and a serverless database . . . 15

4. Azure SQL Managed Instance . . . 16

4.1 What you get . . . 16

4.2 Architecture . . . 16

4.3 Networking . . . 17

4.4 Service tiers and hardware . . . 19

4.5 Management operations and maintenance . . . 20

5. Security . . . 21

5.1 Network security . . . 21

5.2 Authentication and authorization . . . 21

5.3 Encryption: in transit, at rest and in use . . . 22

5.4 Protecting data inside the database . . . 22

5.5 Threat protection and auditing . . . 23

5.6 Security checklist . . . 23

6. Backup and Restore . . . 24

6.1 How automatic backups work . . . 24

6.2 Retention types . . . 24

6.3 Backup storage redundancy . . . 25

6.4 Restore options . . . 25

6.5 Native backups on Managed Instance . . . 25

6.6 Commands you will actually use . . . 26

6.7 Backup and restore best practices . . . 26

7. Monitoring and Performance . . . 27

7.1 The monitoring toolbox . . . 27

7.2 Key metrics to watch . . . 27

7.3 Query Store and automatic tuning . . . 28

7.4 Sending logs to Log Analytics . . . 28

7.5 Starter alert set . . . 28

7.6 Quick troubleshooting guide . . . 29

8. High Availability and Disaster Recovery . . . 30

8.1 Key terms . . . 30

8.2 Built-in high availability by tier . . . 30

8.3 Zone redundancy - surviving a data-centre failure . . . 30

8.4 Disaster recovery options . . . 31

8.5 Auto-failover groups . . . 31

8.6 Application readiness for failover . . . 33

8.7 Choosing the right HA/DR design . . . 33

9. Migration to Azure SQL . . . 34

9.1 Phase 1: Discover and assess . . . 34

9.2 Phase 2: Choose the target and the method . . . 35

9.3 Phase 3: Migrating to Managed Instance . . . 35

9.4 Migrating to Azure SQL Database . . . 36

9.5 Phase 4: Validate and optimise . . . 37

9.6 Common pitfalls and how to avoid them . . . 37

9.7 Sample cutover runbook (online migration) . . . 38

Appendix A: One-page Cheat Sheet . . . 39

Appendix B: Glossary . . . 39

Appendix C: Where to Learn More . . . 40

How to Use This Guide

This guide explains Azure SQL in plain language, with diagrams for every architecture. It is organised in the order most people learn the topic: first the cloud basics, then the products, then how to secure, protect, run and move them.

Chapter	You will learn	Read it if you...
1. Azure Basics	Regions, zones, resource hierarchy, identity, networking blocks	are new to Azure
2. The Azure SQL Family	VM vs Managed Instance vs SQL Database, and how to choose	must pick a platform
3. Azure SQL Database	Logical servers, tiers, serverless, Hyperscale, connectivity	build cloud-native apps
4. Managed Instance	Virtual cluster, subnet and endpoints, tiers, hardware, operations	migrate SQL Server estates
5. Security	Network, identity, encryption, masking, Defender, auditing	own security and compliance
6. Backup and Restore	PITR, LTR, redundancy, restore scenarios, native backups	own data protection
7. Monitoring	Metrics, logs, Query Store, tuning, alerts, troubleshooting	run production systems
8. HA and DR	Zones, failover groups, geo-restore, RPO/RTO, runbooks	design resilience
9. Migration	Assessment, methods, MI link, LRS, cutover, validation	move databases to Azure

A note on accuracy

Azure SQL changes quickly. For example, the Next-gen General Purpose tier for Managed Instance became generally available in late 2025. Numbers such as limits, SLAs, and supported versions are given as typical values to help you understand the design; always confirm them in Microsoft's current documentation before you size, price or sign off an architecture.

How the boxes work

Blue boxes explain ideas in simple words, green boxes give good practice, amber boxes warn about common traps, and purple boxes summarise what to remember.

1. Azure Basics

Before we dive into databases, we need a small mental map of Azure itself. Every Azure SQL service sits on top of the same ideas: where it runs (regions and zones), how it is organised (subscriptions and resource groups), who may use it (identity), and how it connects (networking).

1.1 What is Azure and what is 'cloud'?

Microsoft Azure is a collection of 200+ services (compute, storage, networking, databases, AI, security) that you rent over the internet and pay for as you use them. Instead of buying servers and hiring people to rack, patch and power them, you ask Azure for a resource and it appears in minutes.

Cloud services come in three classic flavours, which decide **how much work stays with you**:

Model	What you get	You manage	Azure SQL example
IaaS **Infrastructure as a Service**	Virtual machines, disks, networks	OS, patching, SQL Server install, backups, HA design	SQL Server on an Azure VM
PaaS **Platform as a Service**	A ready-to-use platform; engine and OS are run for you	Your data, schema, queries, users, security settings	Azure SQL Database, SQL Managed Instance
SaaS **Software as a Service**	A complete application	Only your own data and users	Microsoft 365, Power BI

> **In simple words**
> Think of housing. **IaaS** is renting an empty house (you bring furniture, fix the plumbing). **PaaS** is a serviced apartment (furniture, cleaning and repairs included; you just live in it). **SaaS** is a hotel room (just walk in). Azure SQL Database and Managed Instance are the serviced apartments of the database world.

1.2 The shared responsibility model

In the cloud, security and operations are split between you and Microsoft. The more 'managed' the service, the more Microsoft handles. This table shows who does what for the three ways of running SQL Server.

Task	On-premises	SQL on Azure VM	Managed Instance	SQL Database
Data centre, power, hardware	You	Microsoft	Microsoft	Microsoft
Operating system patching	You	You	Microsoft	Microsoft
SQL engine patching / upgrades	You	You	Microsoft	Microsoft
Backups	You	You	Microsoft (automatic)	Microsoft (automatic)
High availability setup	You	You	Built in	Built in
Indexing, query tuning, schema	You	You	You (+ automatic tuning)	You (+ automatic tuning)
Users, permissions, data classification	You	You	You	You
Your data and app design	You	You	You	You

1.3 Azure global infrastructure

Azure runs in many **geographies** (a country or area with a data-residency promise). Each geography has one or more **regions** (a set of data centres connected by a low-latency network). Many regions have **availability zones** and most regions are **paired** with another region for disaster recovery.

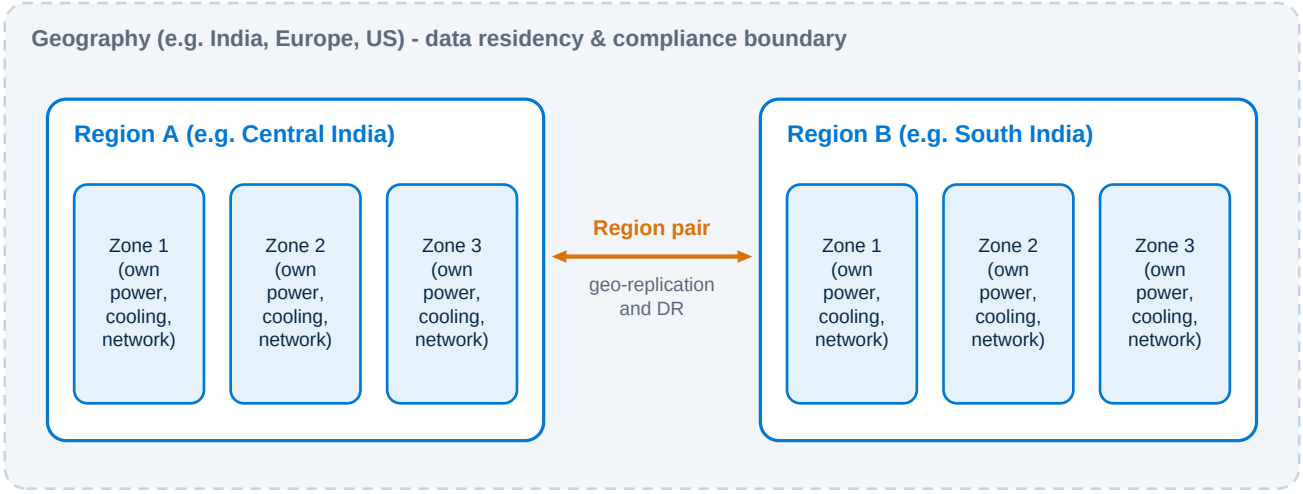


Figure 1.1 - Geography, regions, region pair and availability zones

- **Region** - where you deploy, e.g. Central India, South India, West Europe. Latency, price and available features vary by region.
- **Availability Zone (AZ)** - physically separate data centres inside one region, each with independent power, cooling and networking. Spreading a database over zones protects against a data-centre failure.
- **Region pair** - two regions in the same geography, hundreds of kilometres apart. Used for geo-replication and disaster recovery. Always pick your DR region deliberately.

1.4 How Azure organises resources

Everything you create is a **resource** (a SQL server, a virtual network, a storage account). Resources live in **resource groups**, which live in **subscriptions**. All requests go through **Azure Resource Manager (ARM)**, the single control layer used by the portal, CLI, PowerShell, Bicep, Terraform and APIs.

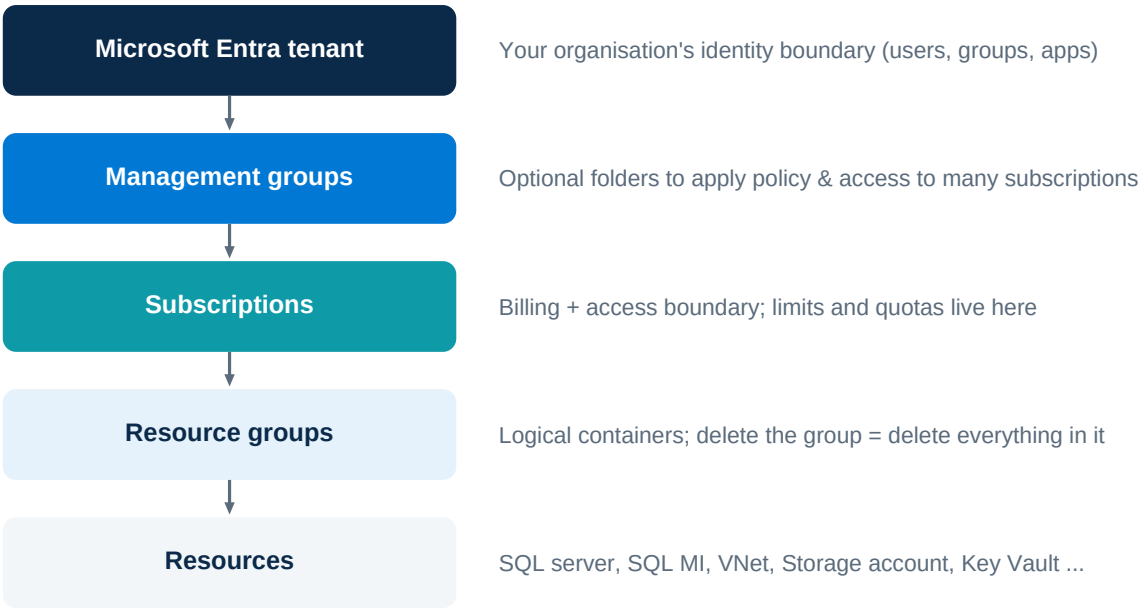


Figure 1.2 - Azure management hierarchy

- **Tags** - name/value labels (Environment=Prod, Owner=Finance) for cost reporting and automation.
- **Azure RBAC** (role-based access control) - who can do what on which scope. Example roles: Reader, Contributor, *SQL DB Contributor*, *SQL Managed Instance Contributor*, *SQL Security Manager*.
- **Azure Policy** - guard-rails such as 'SQL servers must have auditing on' or 'only allow resources in India regions'.
- **Resource locks** - 'Delete' or 'Read-only' locks to protect production servers from accidents.

1.5 Identity: Microsoft Entra ID

Microsoft Entra ID (formerly Azure Active Directory) is Azure's identity service. It holds users, groups, applications and **managed identities** (identities given to Azure resources so they can authenticate without passwords). Azure SQL integrates with Entra ID for sign-in, multi-factor authentication (MFA), conditional access and group-based permissions. It is the recommended way to authenticate to SQL.

1.6 Building blocks you will meet in Azure SQL

Building block	What it is	Why it matters for SQL
Virtual Network (VNet) and subnet	Your private network in Azure, split into address ranges	Managed Instance lives inside a VNet subnet. SQL Database can be reached privately via Private Link.
Network Security Group (NSG)	Allow/deny rules on ports and IPs	Controls traffic to and from MI's subnet.
Route table (UDR)	Custom routes for traffic	Required (and managed) for the MI subnet.
Private Endpoint / Private Link	A private IP in your VNet that maps to a PaaS service	Lets apps reach SQL without crossing the public internet.
VPN Gateway / ExpressRoute	Encrypted tunnel / private circuit to on-premises	Connect on-prem apps and servers to Azure SQL.
Azure Key Vault	Store for keys, secrets and certificates	Holds customer-managed TDE keys and Always Encrypted keys.
Storage account	Blob/file storage	Backups, audit logs, native backup files for migration.
Azure Monitor / Log Analytics	Metrics, logs and alerts	Central monitoring for SQL resources.

1.7 Ways to work with Azure

- **Azure portal** - web interface, great for learning and one-off tasks.
- **Azure CLI** (`az`) and **Azure PowerShell** - scripts and automation.
- **Infrastructure as Code** - Bicep, ARM templates or Terraform, so environments are repeatable and reviewed like code.
- **Tools for data work** - SQL Server Management Studio (SSMS), the MSSQL extension for VS Code, and sqlcmd.

2. The Azure SQL Family

'Azure SQL' is not one product. It is a family of options built on the same SQL Server engine, offered at different levels of control. Choosing the right member is the first big design decision.

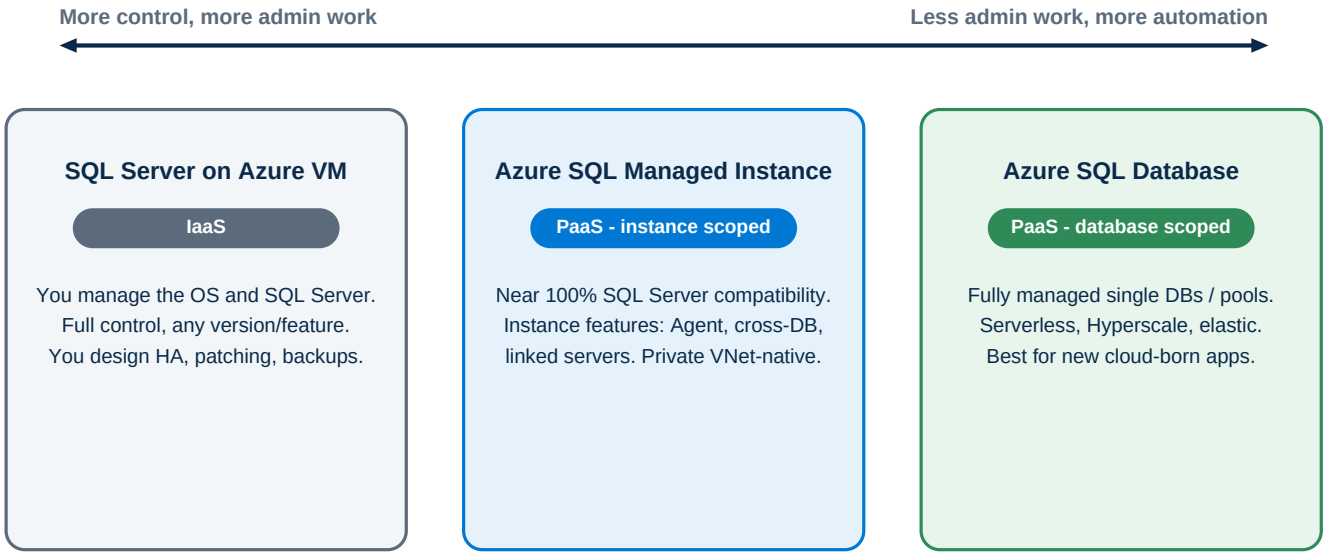


Figure 2.1 - The Azure SQL family on the control vs. management spectrum

2.1 Meet the three members

SQL Server on Azure Virtual Machines (IaaS)

A normal Windows or Linux VM with SQL Server installed. You get the whole instance and the OS. Choose it when you need something PaaS does not allow: an old SQL Server version, custom OS-level software, SSIS/SSRS/SSAS on the same box, or full control of file placement.

Azure SQL Managed Instance (PaaS, instance-scoped)

A managed 'SQL Server instance' in the cloud. It offers almost every instance-level feature (SQL Agent, cross-database queries, Service Broker, CLR, linked servers, Database Mail) while Microsoft runs patching, backups and HA. It is built for **lift-and-shift of existing SQL Server estates** with minimal code change.

Azure SQL Database (PaaS, database-scoped)

A fully managed database service. You deploy a database (or a pool of databases) and never see the server or OS. It scales fast, offers serverless auto-pause and Hyperscale up to 128 TB, and is the natural home for **new, cloud-native applications** and multi-tenant SaaS products.

Also in the family: Azure Arc-enabled SQL

Azure Arc extends Azure management (inventory, patching, Defender, best-practice assessment, and migration assessment) to SQL Server running outside Azure - on-premises or in other clouds. It is a management layer, not a hosting option.

2.2 Side-by-side comparison

Feature	SQL on Azure VM	SQL Managed Instance	SQL Database
Service model	IaaS	PaaS	PaaS
Scope of a deployment	Whole server / OS	Instance with many databases	One database (or pool)
SQL Server compatibility	100%	Near 100%	High; some instance features missing
SQL Server Agent	Yes	Yes	No (use Elastic Jobs, Automation, Functions)
Cross-database queries	Yes	Yes	Limited (elastic query)
Native backup / restore (.bak)	Yes	Yes (copy-only, from URL)	No (BACPAC or automatic backups)
Network model	In your VNet	Injected into your VNet (private by default)	Public endpoint with firewall, or Private Link
Automatic backups and PITR	No (you set up)	Yes	Yes
Built-in HA	You build (AG/FCI)	Yes (99.99% SLA)	Yes (up to 99.995-99.99% SLA)
Max database size	Disk-limited	Up to instance size (up to ~32 TB)	Up to 128 TB (Hyperscale)
Serverless / auto-pause	No	No (start/stop supported)	Yes (GP, Hyperscale)
Best for	Special OS / version needs	Lift-and-shift migration	New apps, SaaS, elastic scale

2.3 How to choose

If your situation is...	Pick
Existing SQL Server estate, uses SQL Agent, cross-DB joins, linked servers, you want minimal code change and no server admin	**Managed Instance**
Building a new app, want per-database scaling, serverless, or very large databases	**SQL Database** (Hyperscale for large/fast-growing)
Many small databases with unpredictable, spiky usage (SaaS tenants)	**SQL Database elastic pool**
Need OS access, third-party agents, unsupported SQL version, or SSRS/SSIS/SSAS on the same host	**SQL Server on Azure VM**
Strict private-only networking from day one with a legacy app	**Managed Instance** (VNet-native)

What all PaaS Azure SQL options share

Always-current database engine - no version upgrades to plan, no OS patches to apply.
Built-in high availability, automated backups, point-in-time restore.
Intelligent features: Query Store, automatic tuning, Microsoft Defender for SQL.
Pay-as-you-go or reserved capacity; Azure Hybrid Benefit lets you reuse existing SQL Server licences with Software Assurance.

3. Azure SQL Database

Azure SQL Database is the most 'cloud-native' member of the family: a fully managed relational database where Microsoft operates everything below your data. This chapter explains what you get, how it is built and how to choose a tier.

3.1 The logical server and databases

An Azure SQL Database lives under a **logical server**. The logical server is *not* a VM; it is an administrative anchor that gives you a DNS name (`myserver.database.windows.net`) and holds shared settings: firewall rules, administrators, auditing, Defender for SQL, private endpoints and failover groups. Databases under it are independent, and each can have a different tier and size.

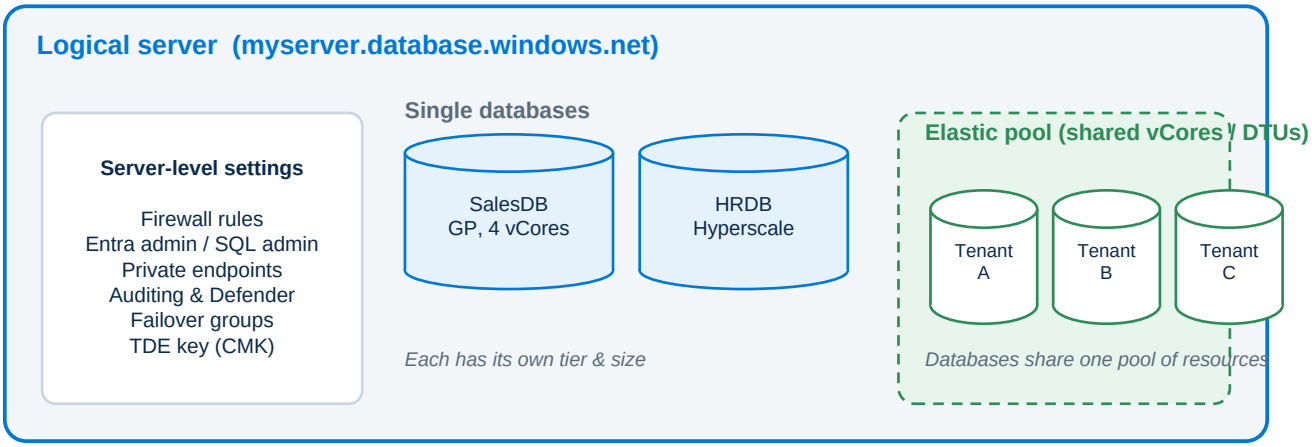


Figure 3.1 - A logical server with single databases and an elastic pool

3.2 Deployment options

Option	What it is	Choose it when
Single database	One isolated database with its own compute and storage	A dedicated database per application or when workloads are predictable
Elastic pool	A shared bucket of vCores/DTUs used by many databases	Many databases whose peaks do not happen at the same time (typical SaaS multi-tenant pattern)
Hyperscale	A tier (single DB or pool) with a distributed storage design up to 128 TB	Large or fast-growing databases, very fast backup/restore, many read replicas
Serverless (compute option)	Auto-scales vCores between a min and max and can pause when idle	Intermittent or unpredictable usage, dev/test

3.3 Purchasing models: DTU vs vCore

A **DTU** (Database Transaction Unit) is a blended bundle of CPU, memory and I/O. It is simple, but you cannot see or tune the parts. The **vCore** model separates compute, memory and storage so you pay for what you pick. It is the recommended model for most new deployments because it supports the newest features (serverless, Hyperscale, zone redundancy, Azure Hybrid Benefit and reservations).

	DTU model	vCore model
Tiers	Basic, Standard, Premium	General Purpose, Business Critical, Hyperscale

	DTU model	vCore model
Sizing	A single bundle (e.g. S3, P2)	Choose vCores, memory (by hardware) and storage separately
Serverless / Hyperscale	Not available	Available
Hybrid Benefit / reservations	Not available	Available
Best for	Small, simple workloads	Most workloads, especially anything that must grow

3.4 vCore service tiers in depth

The service tier decides the **storage architecture**, speed, high-availability design and size limits. Study the three pictures below; they explain most of the differences in the table that follows.

General Purpose - separate compute and storage

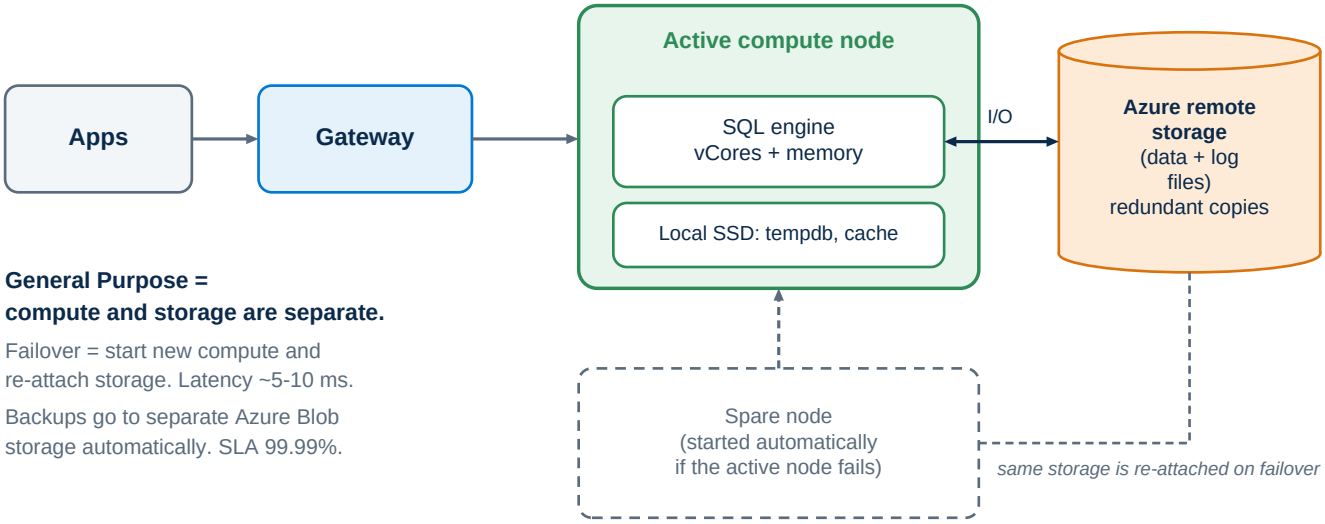


Figure 3.2 - General Purpose: remote storage, replaceable compute

Business Critical - local SSD with an Always On group

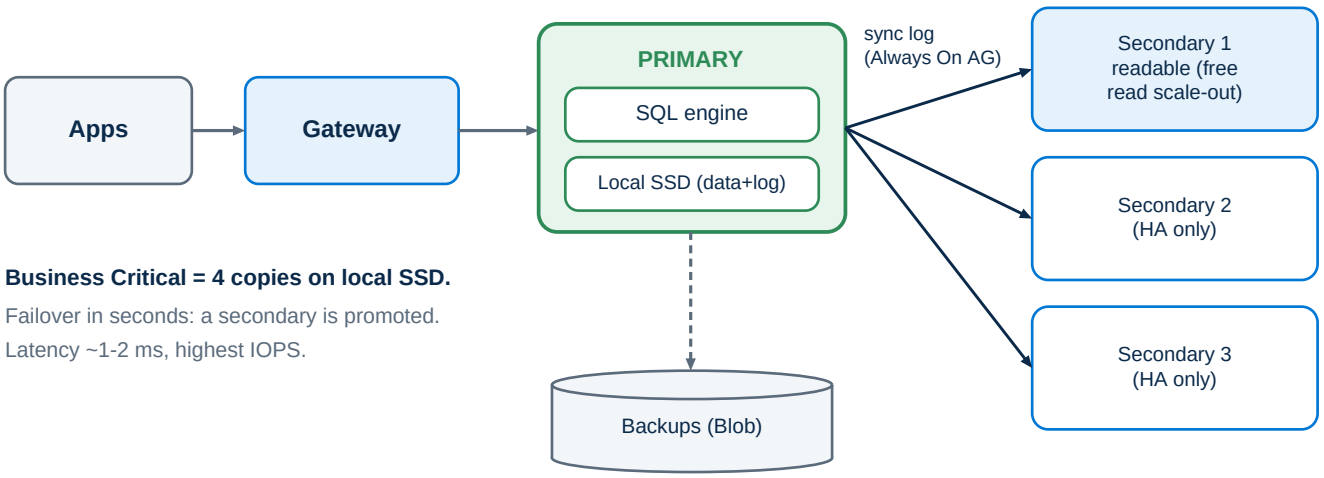


Figure 3.3 - Business Critical: four replicas, one readable

Hyperscale - distributed page servers

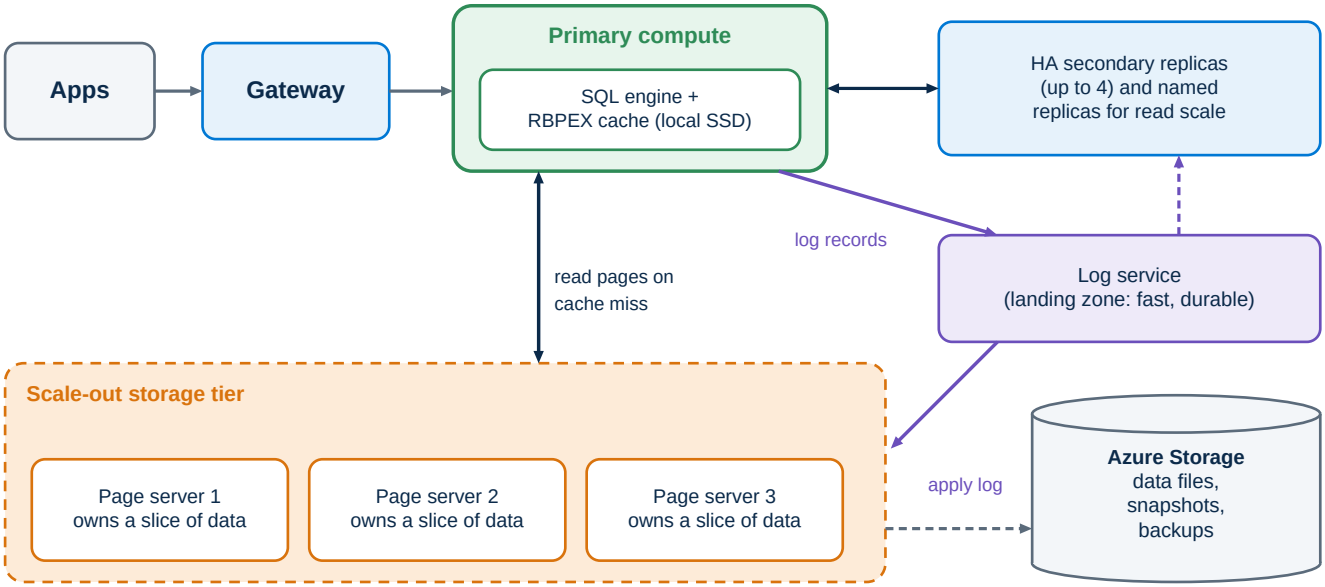


Figure 3.4 - Hyperscale: compute, log service and page servers

In Hyperscale, the database is split into pages held by many **page servers**. Compute nodes keep a hot cache on local SSD (called RBPEX) and fetch other pages on demand. Because backups are storage snapshots, backup and restore time barely depends on database size, and you can add **named replicas** to scale reads independently of the primary.

	General Purpose	Business Critical	Hyperscale
Typical use	Most business apps, dev/test	Low-latency OLTP, high IOPS, strict HA	Very large or fast-growing DBs, mixed OLTP + analytics
Storage design	Remote storage + stateless compute	Local SSD, 4 replicas	Page servers + log service, local SSD cache
Max database size	Up to 4 TB	Up to 4 TB	Up to 128 TB
Storage latency	~5-10 ms	~1-2 ms	~1-2 ms (cache) / 5-10 ms (remote)
HA mechanism	Replace compute, re-attach storage	Automatic failover to a synchronous secondary	Failover to HA secondary; page servers are redundant
Read scale-out	No	1 readable secondary included	Up to 4 HA replicas + up to 30 named replicas
Zone redundancy	Optional	Optional	Optional
Serverless	Yes	No	Yes
Backup / restore speed	Depends on size	Depends on size	Snapshot based, minutes regardless of size

Limits such as maximum vCores, memory per vCore and IOPS change over time and vary by hardware series. Always confirm current numbers in the Microsoft 'resource limits' pages before sizing.

3.5 Provisioned vs serverless compute

Provisioned compute is always on with a fixed number of vCores and is billed hourly. **Serverless** lets you set a *minimum* and *maximum* vCore range. The database scales within that range as load changes and can **auto-pause** after a period of inactivity, during which you pay only for storage. The first connection after a pause triggers a resume that takes a short while.

	Provisioned	Serverless
Billing	Per hour for the vCores you reserved	Per second for vCores actually used (plus storage)
Scaling	You change the size manually	Automatic between min and max vCores
Pause when idle	No	Yes (configurable delay)
Latency after idle	None	Brief delay while the database resumes
Good for	Steady, 24x7 workloads	Spiky, intermittent, dev/test workloads

3.6 Elastic pools - a worked example

Imagine 20 customer databases. Each one needs 4 vCores at its busiest, but only for about an hour a day, and at different times. Buying 20 x 4 = 80 vCores wastes money. An elastic pool of, say, 12 vCores lets each database burst up to its own maximum while the pool as a whole stays within 12 vCores. You set a **per-database minimum and maximum** so no single tenant can starve the others (the 'noisy neighbour' problem).

> **Good practice**
> Use a pool when the *sum of peaks* is much larger than the *peak of the sum*. Monitor pool CPU and storage; if all tenants peak together, a pool will not save money.

3.7 Connectivity architecture

Clients never connect straight to a database node. They first reach a **gateway** at the server's DNS name on port 1433. What happens next depends on the **connection policy**:

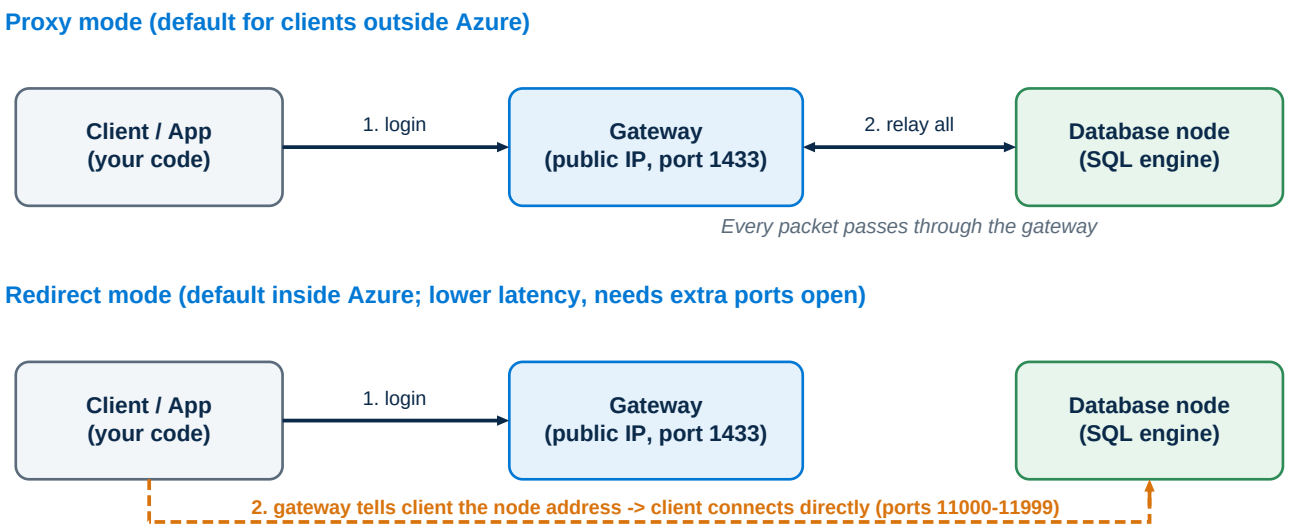


Figure 3.5 - Proxy vs Redirect connection policy

- **Default** policy = Redirect for connections coming from inside Azure, Proxy for connections from outside Azure.
- **Redirect** gives lower latency and higher throughput but requires outbound ports 11000-11999 (and 14000-14999 for the dedicated admin connection) to be open between the client and Azure.

- **Proxy** only needs port 1433 but adds one hop.

Ways to reach SQL Database privately or securely

Method	How it works	Notes
Server firewall (IP rules)	Allow specific public IP ranges	Simplest. Also database-level rules. Use 'Allow Azure services' carefully; it admits all Azure tenants' traffic.
Virtual network rules (service endpoints)	Allow a VNet subnet to reach the public endpoint	Traffic stays on Microsoft backbone, but the server still has a public address.
Private endpoint (Private Link)	A private IP in your VNet maps to the server	Recommended for production. You can then disable public network access. Needs private DNS zone `privatelink.database.windows.net`.
Deny public network access	Turns off the public endpoint entirely	Combine with Private Link.
Minimum TLS version	Rejects old TLS clients	Use TLS 1.2 or higher.

3.8 Quick start: create a server and a serverless database

```
# Create a logical server with Entra-only authentication
az sql server create --name myserver --resource-group rg-sql --location centralindia \
  --enable-ad-only-auth --external-admin-principal-type Group \
  --external-admin-name "SQL-Admins" --external-admin-sid <entra-group-object-id>

# Create a General Purpose serverless database (0.5-4 vCores, pause after 60 minutes)
az sql db create --resource-group rg-sql --server myserver --name salesdb \
  --edition GeneralPurpose --family Gen5 --compute-model Serverless \
  --min-capacity 0.5 --capacity 4 --auto-pause-delay 60
```

Watch out

Replace the placeholder values with your own. Parameter names can change between Azure CLI versions, so run `az sql db create --help` if a flag is not recognised.

4. Azure SQL Managed Instance

Managed Instance (MI) gives you a whole SQL Server *instance* as a managed service, placed inside your own virtual network. It is the PaaS option with the highest compatibility, designed to make 'move my SQL Server to Azure' as painless as possible.

4.1 What you get

- **Near 100% SQL Server compatibility** - the same engine as the latest SQL Server, kept up to date by Microsoft.
- **Instance-level features** - SQL Server Agent jobs, Database Mail, linked servers, cross-database queries and transactions, CLR, Service Broker, server-level logins and triggers, instance time zone and collation.
- **Many databases per instance** - up to 100 in the General Purpose and Business Critical tiers; up to 500 in Next-gen General Purpose.
- **Native VNet integration** - a private IP address inside your network; no public exposure unless you turn it on.
- **Fully managed operations** - patching, automated backups, point-in-time restore, built-in HA, and a 99.99% SLA.
- **Hybrid features** - Managed Instance link (live replication with SQL Server) and native restore from `.bak` files in Blob Storage.

> **What MI does not do**
> You cannot log in to the server or OS. Some Windows/OS-level features are unavailable (for example FILESTREAM/FileTable, Stretch Database, xp_cmdshell). Always run an assessment before migrating.

4.2 Architecture

The virtual cluster

When you create the first MI in a subnet, Azure builds a **virtual cluster** inside that subnet: a set of isolated, Azure-managed virtual machines that host your instance, the gateway and management nodes. Multiple instances can share one subnet (and therefore one virtual cluster). You never log on to these machines; you only see the instance endpoint.

Think of the virtual cluster as a private, fenced-off wing of the Azure data centre that has been plugged into *your* network. That is why MI needs its own dedicated subnet, and why creating the first instance takes longer than creating later ones.

Service tier architectures

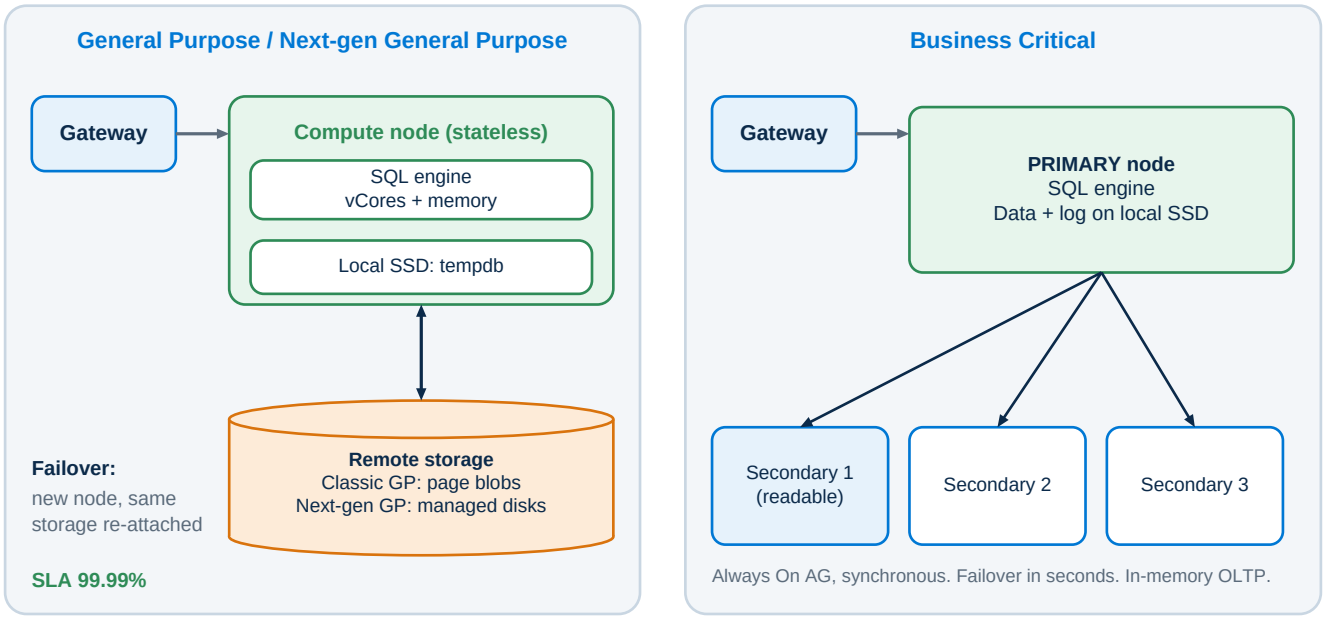


Figure 4.1 - Managed Instance tier architectures

- **General Purpose**: stateless compute plus remote storage (page blobs). If the node fails, Azure starts a new node and re-attaches the same storage.
- **Next-gen General Purpose**: the same tier family with a **modernised storage layer on managed disks**. It brings better latency and throughput, an IOPS slider, flexible memory, more vCores, up to 32 TB storage and up to 500 databases, with the same baseline price as General Purpose. It can be enabled on new or existing GP instances.
- **Business Critical**: four replicas (one primary, three secondaries) on local SSD in an Always On availability group. One secondary is readable for free read scale-out. It adds In-Memory OLTP and the lowest latency.

4.3 Networking

Networking is the part of MI that surprises people most, so we cover it carefully. MI is **VNet-injected**: it is deployed *into* a subnet you own, rather than reached through a shared public service endpoint.

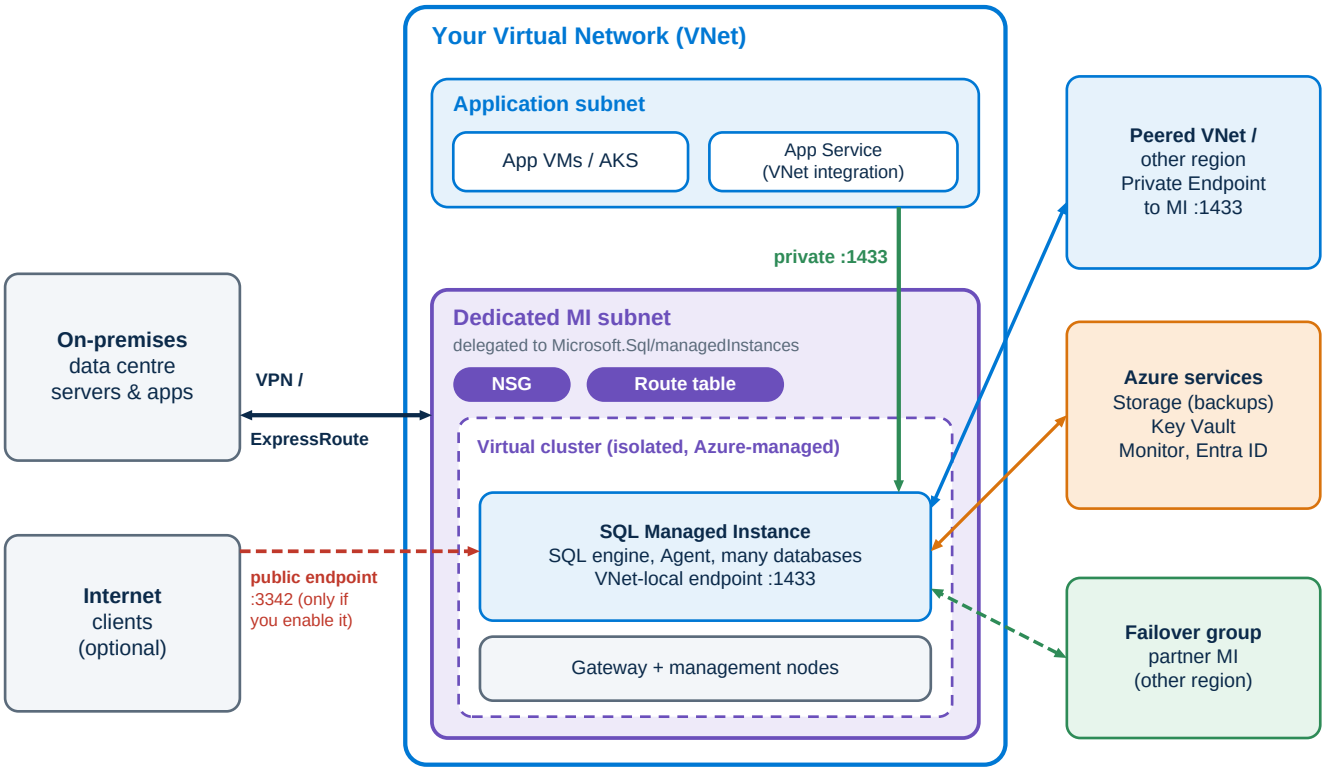


Figure 4.2 - Managed Instance inside a virtual network and the ways to reach it

Subnet rules

- **Dedicated subnet** - only Managed Instances (and nothing else) can live in the subnet.
- **Delegation** - the subnet must be delegated to `Microsoft.Sql/managedInstances`. This lets Azure manage the network policy inside it.
- **NSG and route table** - both must be attached. With *service-aided subnet configuration*, Azure adds and maintains the mandatory rules and routes. **Do not delete or alter these managed rules.**
- **Size** - plan generously. Each instance and its supporting nodes consume IP addresses, and growth in vCores or tier can require more. Use at least a /27 range and prefer something larger for production; use Microsoft's sizing guidance to calculate exactly.
- **Address space** - choose a range that does not overlap with on-premises networks if you will connect them.

Endpoints and ports

Endpoint	Address form	Port	Notes
VNet-local (default)	`name.<dnszone>.database.windows.net`	1433 (Proxy) or 1433 + 11000-11999 (Redirect)	Private IP; reachable from the VNet, peered VNets and on-premises
Public endpoint (optional)	`name.public.<dnszone>.database.windows.net`	3342	Must be enabled on the instance and allowed in the NSG. Protect with NSG source IP rules.
Private endpoint (optional)	Private IP in another VNet or subnet	1433	Lets other networks reach MI privately without peering all subnets

Common connectivity scenarios

From where?	How to connect	What to set up
App in the same VNet	Private IP, port 1433	NSG allows app subnet to MI subnet
App in another VNet / region	VNet peering (including global peering) or private endpoint	Non-overlapping address spaces; DNS resolution of the MI name
On-premises servers and users	Site-to-site VPN or ExpressRoute	Routes, DNS forwarding to resolve the instance name, firewall rules
Azure App Service, Functions	VNet integration into a subnet that can reach MI	Regional VNet integration or a private endpoint
Internet / partner (only if unavoidable)	Public endpoint on port 3342	Enable endpoint, restrict source IPs in NSG, require Entra auth and TLS

DNS matters

The instance name resolves to a private IP inside your VNet. Applications that sit outside the VNet (on-premises, other clouds) need DNS forwarding or conditional forwarders to resolve it. Many 'cannot connect' problems are really DNS problems.

Outbound access

MI must reach Azure management, storage and identity services. Azure manages these rules for you through service tags in the NSG and route table. Do not add a firewall rule or user-defined route that blocks them, or management operations will fail.

4.4 Service tiers and hardware

	General Purpose	Next-gen General Purpose	Business Critical
Best for	Most business apps; budget-aware	GP price band but needs more IOPS, memory, storage or databases	Mission-critical OLTP, lowest latency, read scale-out
Storage design	Remote storage (page blobs)	Remote storage (managed disks)	Local SSD
Storage latency	~5-10 ms	Lower than classic GP, tunable IOPS slider	~1-2 ms
Max instance storage	Up to 16 TB	Up to 32 TB	Up to 16 TB (hardware dependent)
Max databases	100	500	100
Memory model	Fixed per vCore	Flexible memory (choose amount)	Fixed per vCore
Readable secondary	No	No	Yes (included)
In-Memory OLTP	No	No	Yes
HA design	Compute re-start + remote storage	Compute re-start + remote storage	Always On AG, 4 replicas
Zone redundancy	Optional	Optional	Optional
Relative price	1x	Same baseline as GP (IOPS slider extra)	About 2.7x GP

Values reflect Microsoft documentation at the time of writing. Next-gen GP became generally available in November 2025. Check the current resource-limits page before final sizing.

Hardware series

Series	Memory per vCore	Typical use
Standard-series (Gen5)	5.1 GB	General workloads
Premium-series	7 GB	Newer CPUs, better performance for demanding workloads
Memory-optimized premium-series	13.6 GB	Memory-heavy workloads, large caches, analytics-like queries

Ways to reduce cost

- **Azure Hybrid Benefit** - reuse existing SQL Server licences with Software Assurance and pay only for compute (GP and BC).
- **Reservations** - commit to 1 or 3 years of vCores for a discount.
- **Stop/start** - stop dev/test instances when not in use so you do not pay for compute.
- **Right-size** - note that you are charged for the *reserved* storage you set, not just what is used.

4.5 Management operations and maintenance

- **Create / scale / change tier** are asynchronous *management operations*. They can take from minutes to hours, especially the first instance in a new subnet and Business Critical changes that must re-seed replicas. The instance stays online; there is a short connection drop at the end when the new compute takes over.
- **Maintenance windows** - schedule platform maintenance (default window and optional alternatives) and enable advance notifications so planned updates do not collide with business peaks.
- **Instance pools** - for small workloads, pools let several small instances share the same virtual cluster at lower cost.
- **Update policy** - choose the engine update policy (always-up-to-date, or aligned to a specific SQL Server release) to control feature and compatibility changes.

Quick start: create a Managed Instance

```
az sql mi create --name mi-prod-01 --resource-group rg-sql --location centralindia \
  --subnet /subscriptions/<sub>/resourceGroups/rg-net/providers/Microsoft.Network/\
virtualNetworks/vnet-hub/subnets/snet-mi \
  --edition GeneralPurpose --family Gen5 --capacity 8 --storage 256GB \
  --admin-user sqladmin --admin-password '<strong-password>'
```

In practice, prefer Bicep or Terraform, Entra-only authentication, and create the NSG, route table and delegated subnet first. The first deployment in a new subnet can take several hours, so plan ahead.

5. Security

Security in Azure SQL is layered, like the walls and locks of a castle. No single control is enough; you combine network isolation, strong identity, threat detection and data encryption so that a failure in one layer does not expose your data.

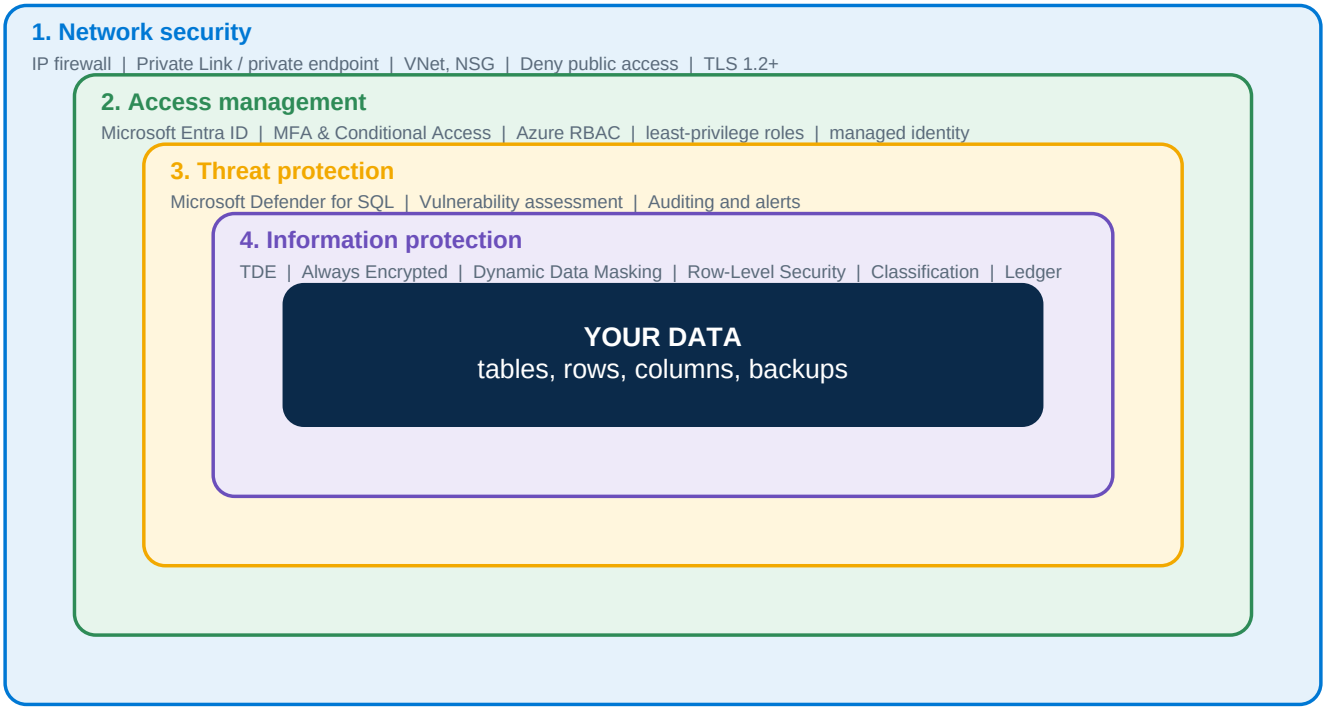


Figure 5.1 - Defence in depth for Azure SQL

5.1 Network security

Control	SQL Database	Managed Instance
Default exposure	Public endpoint, blocked by firewall until you allow IPs	Private IP in your VNet; no public access by default
Firewall	Server and database IP rules, VNet rules	NSG on the MI subnet (source/port rules)
Private access	Private endpoint (Private Link)	VNet-local endpoint, peering, private endpoint
Disable public access	Yes ('Public network access: Disabled')	Public endpoint is off unless you enable it
TLS	Minimum TLS version setting (use 1.2+)	Minimum TLS version setting (use 1.2+)
Outbound control	Outbound networking restrictions (allow-list FQDNs)	NSG/UDR within service-managed rules

5.2 Authentication and authorization

Authentication proves who you are. **Authorization** decides what you may do. Azure SQL supports:

- **Microsoft Entra authentication** (recommended) - users, groups, service principals and managed identities, with MFA and Conditional Access, and centralised joiner/mover/leaver control.
- **SQL authentication** - username and password stored in the database. Simple, but weaker. You can switch it off with **Entra-only authentication**.

Authorization works at two layers: **Azure RBAC** for the control plane (who can create, scale or delete the server) and **database permissions** for the data plane (who can read or write tables). They are separate - being an Azure 'Owner' does not automatically let you read the data.

Layer	Examples	Where you set it
Azure RBAC (control plane)	SQL DB Contributor, SQL Managed Instance Contributor, SQL Security Manager	Azure portal / ARM
Server-level (MI)	Logins, sysadmin, SQL Agent roles	T-SQL on the instance
Database-level	db_datareader, db_datawriter, db_owner, custom roles, GRANT/DENY	T-SQL in each database
Row/column-level	Row-Level Security policies, column permissions	T-SQL

```
-- Azure SQL Database: give an Entra group read access (contained user, no server login needed)
CREATE USER [Sales-Analysts] FROM EXTERNAL PROVIDER;
ALTER ROLE db_datareader ADD MEMBER [Sales-Analysts];

-- Managed Instance: create a server-level login for an Entra group
CREATE LOGIN [DBA-Team] FROM EXTERNAL PROVIDER;
ALTER SERVER ROLE sysadmin ADD MEMBER [DBA-Team];
```

Good practice

Give people and apps the **minimum** permission they need. Prefer Entra *groups* over individual users, and use **managed identities** for application access so no password is stored in code or config files.

5.3 Encryption: in transit, at rest and in use

Stage	Feature	How it works
In transit	TLS 1.2+	All connections are encrypted. Clients should validate the certificate (do not enable 'trust server certificate' in production).
At rest	Transparent Data Encryption (TDE)	On by default. Encrypts data files, logs and backups with a Database Encryption Key. Default is a service-managed key; you can use a customer-managed key in Azure Key Vault (BYOK) for control and rotation.
In use	Always Encrypted	Client driver encrypts sensitive columns (SSN, card numbers) before sending. The server never sees plaintext or keys. Secure enclaves allow richer queries on encrypted data.

5.4 Protecting data inside the database

- **Dynamic Data Masking** - shows masked values (like `aXXX@XXXX.com`) to users without the UNMASK permission. Not a security boundary on its own, but a useful guard rail.
- **Row-Level Security (RLS)** - automatically filters rows per user, for example a tenant only sees its own orders.
- **Data Discovery and Classification** - finds columns that look sensitive (PII, financial) and labels them for audit and reporting.
- **Ledger tables** - tamper-evident tables with cryptographic proof, useful for audit and regulated records.

```
-- Mask an email column
ALTER TABLE dbo.Customers ALTER COLUMN Email ADD MASKED WITH (FUNCTION = 'email()');

-- Row-Level Security: each session sees only its tenant's rows
CREATE SCHEMA Security;
GO
CREATE FUNCTION Security.fn_tenant(@TenantId int) RETURNS TABLE WITH SCHEMABINDING AS
  RETURN SELECT 1 AS ok WHERE @TenantId = CAST(SESSION_CONTEXT(N'TenantId') AS int);
GO
CREATE SECURITY POLICY Security.TenantPolicy
  ADD FILTER PREDICATE Security.fn_tenant(TenantId) ON dbo.Orders WITH (STATE = ON);
```

5.5 Threat protection and auditing

Microsoft Defender for SQL adds two things: **Vulnerability Assessment** (scans for misconfigurations such as excessive permissions or unencrypted data and suggests fixes) and **Advanced Threat Protection** (alerts on SQL injection attempts, brute-force logins, unusual access locations and suspicious activity).

Auditing records database events (logins, queries, permission changes) so you can investigate incidents and prove compliance. Send audit logs to an Azure Storage account, a Log Analytics workspace or Event Hubs. Enable it at the server level so every database is covered.

5.6 Security checklist

Done?	Control	Why
[ ]	Use Entra authentication; enable Entra-only auth where possible	Central control, MFA, no shared passwords
[ ]	Use private endpoints; disable public network access	Removes internet exposure
[ ]	Set minimum TLS 1.2; validate certificates in clients	Prevents downgrade and interception
[ ]	Keep TDE on; consider customer-managed keys for regulated data	Encryption at rest and key control
[ ]	Enable Defender for SQL and review recommendations	Detects attacks and weaknesses
[ ]	Enable server-level auditing to Log Analytics or Storage	Investigation and compliance evidence
[ ]	Apply least privilege; use managed identities for apps	Limits damage from stolen credentials
[ ]	Classify sensitive columns; mask or encrypt as needed	Protects PII and financial data
[ ]	Add resource locks and Azure Policy for guard-rails	Prevents accidental or non-compliant changes

6. Backup and Restore

Azure SQL takes backups for you automatically. There is nothing to schedule and no backup software to buy. Your job is to choose how long to keep them, where to store them, and to practise restoring them.

6.1 How automatic backups work

Azure SQL Database and Managed Instance use a standard SQL Server backup pattern, managed by the platform: a weekly **full** backup, **differential** backups about every 12 hours, and **transaction log** backups about every 10 minutes (the exact rate depends on compute size and database activity). Combining these lets Azure rebuild the database as it was at any moment inside your retention window - this is **point-in-time restore (PITR)**.

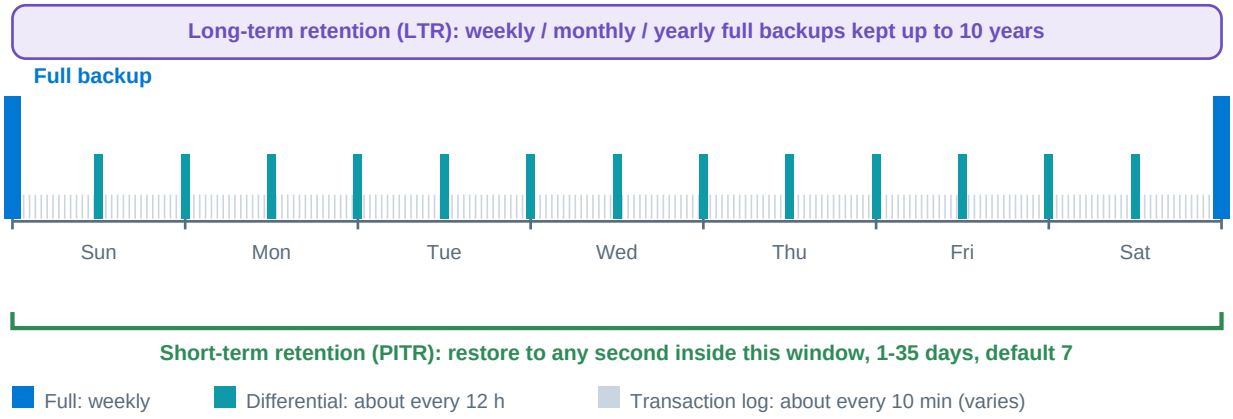


Figure 6.1 - Backup types and retention periods

- Backups are stored in Azure Storage, automatically checked and encrypted (with TDE).
- In Hyperscale, backups are **storage snapshots**, so they are fast and do not slow down the database regardless of size.
- Backups do not use your database's vCores or I/O in a way you need to manage; the platform schedules them.

6.2 Retention types

	Short-term retention (PITR)	Long-term retention (LTR)
Purpose	Recover from mistakes, corruption, bad deployments	Compliance, audits, archives
Granularity	Any point in time (to the second)	Specific weekly, monthly, yearly full backups
Default	7 days	Off until you configure a policy
Range	1 to 35 days	Up to 10 years
Example policy	14 days	Weekly: 4 weeks, Monthly: 12 months, Yearly: 5 years

6.3 Backup storage redundancy

You choose how durable the backup copies are. The choice affects both cost and what you can do if a whole region is lost.

Option	Copies	Survives	Region-failure recovery?
LRS (locally redundant)	3 copies in one data centre	Disk/rack failure	No
ZRS (zone redundant)	3 copies in 3 zones, one region	Data centre (zone) failure	No
GRS (geo-redundant)	LRS in primary + copy in paired region	Region outage	Yes - geo-restore
GZRS (geo-zone redundant)	ZRS in primary + copy in paired region	Zone and region outage	Yes - geo-restore

Good practice

For production, GRS or GZRS is the safe choice because it keeps a backup copy in another region. LRS is usually fine only for dev/test. Backup redundancy is set when you create the resource; check the portal for whether it can be changed afterwards for your service.

6.4 Restore options

A restore **always creates a new database** (it never overwrites the original). You then compare, copy back the data you need, or rename the new database to swap it in.

Scenario	Restore type	Notes
Someone deleted or changed rows by mistake	Point-in-time restore to a new database	Pick a time just before the mistake; copy the rows back
Database was dropped	Restore deleted database	Possible within the PITR retention window
Region outage, no failover group	Geo-restore from geo-redundant backup	Needs GRS/GZRS; RPO up to about 1 hour
Auditor needs a 3-year-old copy	Restore from long-term retention	Needs an LTR policy
Managed Instance: copy DB to another MI	PITR to a different instance	Same region/subscription rules apply; check cross-subscription options
Move DB from on-prem or between MIs	Native copy-only backup and RESTORE FROM URL	Managed Instance only

6.5 Native backups on Managed Instance

Besides automatic backups, MI supports **copy-only** native backups to Azure Blob Storage with T-SQL. They do not break the automatic backup chain and are perfect for moving a database to another instance or for creating a portable copy. SQL Database does not support native backups; use BACPAC export or database copy instead.

```
-- Managed Instance: copy-only full backup to Blob Storage (uses a SAS or managed identity credential)
BACKUP DATABASE SalesDB
TO URL = 'https://mystorage.blob.core.windows.net/backups/SalesDB.bak'
WITH COPY_ONLY, COMPRESSION, STATS = 10;

-- Restore a .bak file from Blob Storage into MI
RESTORE DATABASE SalesDB_Copy
FROM URL = 'https://mystorage.blob.core.windows.net/backups/SalesDB.bak';
```

6.6 Commands you will actually use

```
# Point-in-time restore of an Azure SQL Database to a NEW database
az sql db restore --resource-group rg-sql --server myserver --name salesdb \
  --dest-name salesdb-restored --time "2026-10-28T10:15:00Z"

# Point-in-time restore of a Managed Instance database
az sql midb restore --resource-group rg-sql --managed-instance mi-prod-01 --name SalesDB \
  --dest-name SalesDB_restored --time "2026-10-28T10:15:00Z"

# Change short-term retention to 14 days
az sql db str-policy set --resource-group rg-sql --server myserver --name salesdb --retention-days 14

# Long-term retention: weekly 4 weeks, monthly 12 months, yearly 5 years (take week 1 as yearly)
az sql db ltr-policy set --resource-group rg-sql --server myserver --name salesdb \
  --weekly-retention P4W --monthly-retention P12M --yearly-retention P5Y --week-of-year 1
```

6.7 Backup and restore best practices

- **Define RPO and RTO first.** RPO = how much data you can lose; RTO = how long you can be down. PITR gives an RPO of roughly minutes; restore time (RTO) grows with database size, except in Hyperscale.
- **Test restores** on a schedule (for example quarterly) and time them. An untested backup is only a hope.
- **Match retention to your needs** - raise PITR beyond 7 days if mistakes are noticed late; use LTR for compliance.
- **Choose GRS/GZRS** backup storage when you rely on geo-restore as part of DR.
- **Protect the server** with resource locks and RBAC - deleting a server removes its backups after the deleted-database window.
- **Use BACPAC** only for portability and small databases; it is not a replacement for automatic backups.

7. Monitoring and Performance

Good monitoring answers four questions: **Is it up? Is it healthy? Is it fast enough? Is it about to run out of something?** Azure gives you tools at three levels: the platform (Azure Monitor), the engine (Query Store and DMVs), and the intelligence layer (automatic tuning and alerts).

7.1 The monitoring toolbox

Layer	Tool	What it gives you
Platform metrics	Azure Monitor metrics	CPU %, storage %, log I/O %, workers, sessions, connections, deadlocks. Collected automatically and free.
Resource logs	Diagnostic settings -> Log Analytics, Storage, Event Hubs	Query Store runtime/wait statistics, errors, timeouts, blocks, deadlocks, audit events
Control-plane events	Azure Activity Log	Who scaled, failed over, changed firewall or deleted what, and when
Query performance	Query Store, Query Performance Insight	Top queries by CPU/duration/IO, plan changes, regressions
Engine internals	DMVs and Extended Events	Live sessions, waits, blocking chains, resource stats
Fleet monitoring	Database watcher (newer service)	Detailed, near-real-time telemetry across many databases and instances; check current availability for your target
Alerts	Azure Monitor alerts + action groups	Email, SMS, webhook, Logic App, ITSM when thresholds are crossed

7.2 Key metrics to watch

Metric	Applies to	What it means	Typical alert
cpu_percent / avg_cpu_percent	SQL DB / MI	Compute pressure	> 80% for 15 min
storage_percent, storage_space_used_mb	SQL DB / MI	How full the allocated storage is (MI reports reserved vs used)	> 85%
log_write_percent	SQL DB	Transaction log throughput limit usage	> 80% for 10 min
workers_percent, sessions_percent	SQL DB	Concurrent requests / sessions vs limit	> 70%
connection_failed	SQL DB	Failed logins and connection errors	> 0 sustained
deadlock	SQL DB	Deadlock count	> 0
io_requests, io_bytes_read/written	MI	Storage I/O activity	Compare with tier limit
tempdb log used percent	SQL DB	tempdb log usage	> 80%

7.3 Query Store and automatic tuning

Query Store is on by default in Azure SQL. It records every query's text, plans and runtime statistics over time, like a flight recorder. It lets you answer 'what changed?' when a query suddenly slows down: often a new execution plan that is worse than the old one.

Automatic tuning uses that history to help:

- **Force last good plan** - when a plan regression is detected, force the earlier faster plan and verify the improvement (available on both SQL Database and Managed Instance, on by default).
- **Create index / drop index** - suggests or applies indexes based on workload, then rolls back if performance worsens (SQL Database only).

```
-- Top 10 queries by total CPU (from Query Store)
SELECT TOP 10 qt.query_sql_text,
       SUM(rs.count_executions) AS executions,
       SUM(rs.avg_cpu_time * rs.count_executions) / 1000000.0 AS total_cpu_sec
FROM sys.query_store_query_text qt
JOIN sys.query_store_query q          ON q.query_text_id = qt.query_text_id
JOIN sys.query_store_plan p           ON p.query_id = q.query_id
JOIN sys.query_store_runtime_stats rs ON rs.plan_id = p.plan_id
GROUP BY qt.query_sql_text
ORDER BY total_cpu_sec DESC;

-- Recent resource usage in an Azure SQL Database (15-second samples, about the last hour)
SELECT TOP 20 end_time, avg_cpu_percent, avg_data_io_percent,
       avg_log_write_percent, avg_memory_usage_percent
FROM sys.dm_db_resource_stats ORDER BY end_time DESC;
```

7.4 Sending logs to Log Analytics

Create a **diagnostic setting** on the database or instance and send the categories you need to a Log Analytics workspace. You can then query with KQL, build dashboards and create log-based alerts. Keep an eye on cost: some categories (like query runtime statistics) can be voluminous.

```
// KQL: average CPU per database over the last 24 hours, 5-minute buckets
AzureMetrics
| where ResourceProvider == "MICROSOFT.SQL" and MetricName == "cpu_percent"
| where TimeGenerated > ago(24h)
| summarize avg(Average) by bin(TimeGenerated, 5m), Resource
| render timechart
```

7.5 Starter alert set

Alert	Condition	First response
High CPU	CPU > 80% for 15 minutes	Check Query Store top CPU queries; consider scaling or tuning
Storage nearly full	Storage > 85%	Increase max size (or reserved storage on MI); archive or clean data
Log I/O saturation	log_write_percent > 80%	Batch large writes, tune, or move to a higher tier
Connection failures	connection_failed > threshold	Check firewall/NSG/DNS, credentials, failovers in Activity Log
Deadlocks	deadlock > 0	Review deadlock graphs, indexing and transaction scope
Failover happened	Activity Log event	Verify app retry logic; review cause
Backup / LTR issues	Resource Health or policy alert	Confirm retention policy and storage redundancy

7.6 Quick troubleshooting guide

Symptom	Likely causes	Where to look
Everything is slow	CPU or I/O at limit; bad plan; missing index	CPU/IO metrics, Query Store, automatic tuning recommendations
One query is slow	Plan regression, stale statistics, parameter sniffing	Query Store plan history; force a plan; update statistics
Timeouts / blocking	Long transactions, lock contention	sys.dm_exec_requests, blocking chains, 'Blocks' log
Cannot connect	Firewall/NSG, DNS, private endpoint, TLS, auth method, failover	Activity Log, connection test, DNS check, error number
Intermittent errors (e.g. 40613, 4060)	Transient faults during failover or scaling	Add **retry logic** with backoff in the application
Storage full	Data growth, tempdb or log growth	Storage metric, sp_spaceused, size limits

Always build retry logic into applications

Cloud databases occasionally reconnect during maintenance, scaling or failover. Well-written apps detect transient errors and retry a few times with a short, growing delay. Most modern drivers and libraries (for example the 'configurable retry logic' in Microsoft.Data.SqlClient) support this.

8. High Availability and Disaster Recovery

High availability (HA) keeps your database running through small, common failures such as a failed server or a software crash. **Disaster recovery (DR)** gets you running again after a large, rare event such as the loss of a whole data centre or region. Azure SQL gives you both, built in, but you choose how much protection to buy.

8.1 Key terms

Term	Meaning	Plain-English example
RPO (Recovery Point Objective)	Maximum acceptable data loss, measured in time	'We can lose at most 5 seconds of orders.'
RTO (Recovery Time Objective)	Maximum acceptable downtime	'We must be back within 1 hour.'
SLA	Microsoft's availability promise for the service, as a percentage of time	99.99% means about 52 minutes of allowed downtime per year
Failover	Switching work from a failed copy to a healthy one	Replica in zone 2 becomes primary
Failback	Returning to the original location after recovery	Return to Region A after the outage ends

8.2 Built-in high availability by tier

Tier	How HA works	Typical failover	Availability SLA*
SQL DB General Purpose / Standard (MI General Purpose)	Stateless compute is replaced; storage re-attached (storage already holds redundant copies)	Tens of seconds	99.99%
SQL DB Business Critical / Premium (MI Business Critical)	Always On availability group with 4 replicas on local SSD; automatic promotion of a secondary	Seconds	99.99% (99.995% with zone redundancy on SQL DB)
SQL DB Hyperscale	Failover to HA secondary replicas; page servers and log service are themselves redundant	Seconds with replicas; longer if none	Up to 99.99% (depends on replicas)

**SLA values come from Microsoft's published SLA documents and can change. Check the current 'SLA for Azure SQL' pages before quoting numbers in a contract.*

8.3 Zone redundancy - surviving a data-centre failure

By default, all replicas of your database may sit in the same data centre. Enabling **zone redundancy** spreads them across **availability zones** in the same region, so the loss of one entire zone is handled automatically, with no data loss and no change to your connection string.

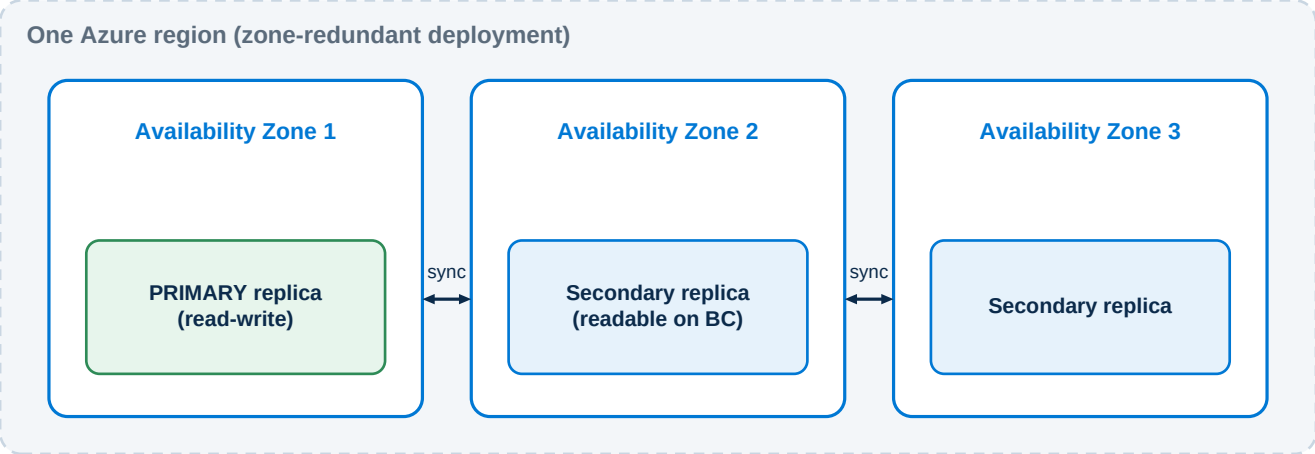


Figure 8.1 - A zone-redundant Business Critical deployment (replicas in three zones)

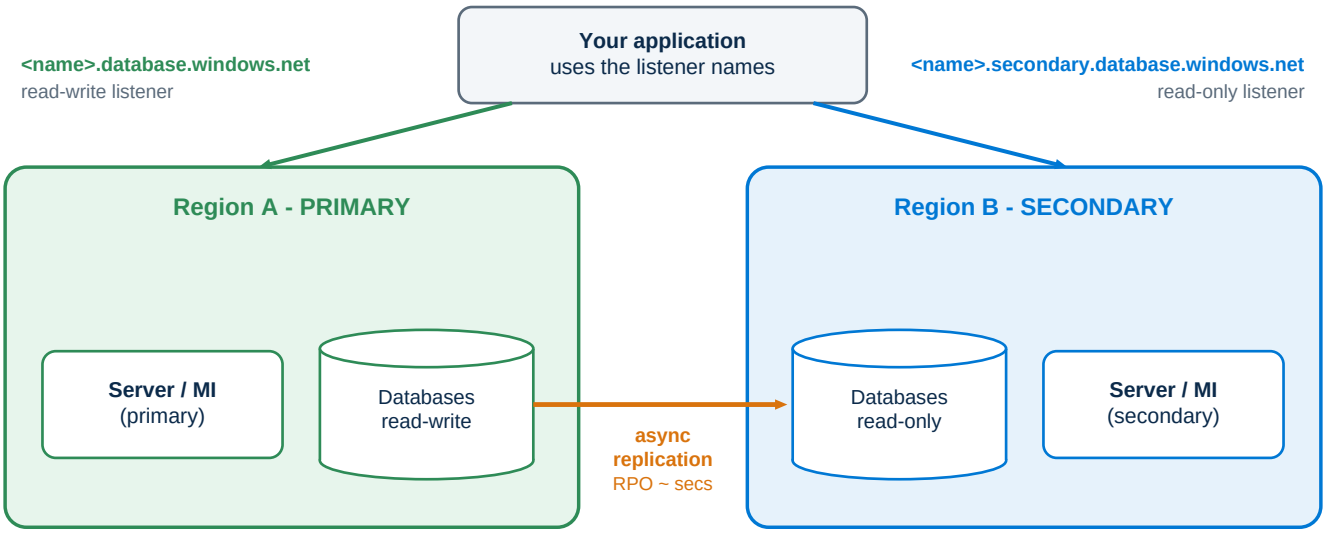
- Available for SQL Database (General Purpose, Business Critical, Hyperscale) and Managed Instance (General Purpose and Business Critical) in regions that have availability zones.
- Adds cost (extra replicas or zone-redundant storage) but gives a stronger SLA for SQL Database Business Critical.
- Combine with **ZRS or GZRS backup storage** so backups also survive a zone failure.
- Zone redundancy protects against *zone* failures only. For a *regional* disaster you also need geo-replication or geo-restore.

8.4 Disaster recovery options

Option	Applies to	RPO (typical)	RTO (typical)	Cost / complexity
Geo-restore from geo-redundant backup	SQL DB, MI	Up to ~1 hour	Hours (can be up to ~12 h or more for large DBs)	Lowest: pay for GRS backup storage only
Active geo-replication	SQL DB	~5 seconds	Seconds to minutes (manual)	Medium: pay for secondary DBs
Auto-failover group	SQL DB, MI	~5 seconds	~1 hour with automatic policy (grace period); minutes if you fail over manually	Medium to high: secondary server/MI
Managed Instance link (to SQL Server or MI)	MI	Seconds	Minutes (manual)	Hybrid DR; needs SQL Server side

8.5 Auto-failover groups

A **failover group** is the recommended DR design. It manages replication between a primary and a secondary in another region, and gives you two stable **listener** names that always point to the current primary (read-write) and the current secondary (read-only). When a failover happens, DNS is switched and your applications keep using the same names.



After failover the roles swap and the same listener names point to the new primary - no connection string change.

Figure 8.2 - Failover group with read-write and read-only listeners

Failover types

- **Planned (manual) failover** - fully synchronises first, so there is **no data loss**. Use it for DR drills, region moves and maintenance.
- **Forced failover** - immediate; used in a real outage. Transactions not yet replicated can be lost (this is your RPO). Be sure you accept that before using it.
- **Automatic failover policy** - Azure fails over by itself after a configurable **grace period** (for example one hour) of unavailability, giving temporary glitches time to recover.

Managed Instance specifics

- The secondary MI must be in a **different region**, with the same or higher tier and compute, and in the same **DNS zone** (so both share a DNS zone ID).
- The two MI subnets must be able to talk to each other (global VNet peering or VPN/ExpressRoute) with **non-overlapping address spaces** and the replication ports open in the NSGs.
- Server-level objects such as **logins, SQL Agent jobs, linked servers and server settings** are not automatically kept in sync; maintain them on both instances (scripts or CI/CD).
- Failover groups for MI operate at the instance level: all user databases fail over together.

```
# SQL Database: create a failover group between two servers and add a database
az sql failover-group create --name fg-sales --resource-group rg-sql \
  --server sql-central --partner-server sql-south \
  --add-db salesdb --failover-policy Manual

# Planned failover: run against the SECONDARY server to make it primary
az sql failover-group set-primary --name fg-sales --resource-group rg-sql --server sql-south

# Managed Instance failover group (instance-level)
az sql instance-failover-group create --name fog-mi --mi mi-central --resource-group rg-sql \
  --location centralindia --partner-mi mi-south --partner-resource-group rg-sql-dr \
  --partner-region southindia --failover-policy Manual
```

Watch out

Command flags can change between CLI versions. Confirm with `--help` or the current docs before running commands in production.

8.6 Application readiness for failover

- Always connect through the **listener name** (failover group endpoint), never through a specific server name.
- Use **retry logic** with exponential back-off for transient errors; connections drop briefly during failover.
- Keep connection pools short-lived enough to pick up the new DNS answer; set DNS caching low.
- Send reporting and read-only workloads to the **read-only listener** or to a readable secondary to reduce primary load.
- Make sure **users, permissions, firewall rules, Key Vault access and monitoring** exist in the secondary region too.
- Automate your **runbook** (scripts or pipelines) and rehearse it at least twice a year.

8.7 Choosing the right HA/DR design

Your requirement	Recommended design
Survive server failures; modest budget	General Purpose with built-in HA (99.99% SLA)
Survive a data-centre (zone) failure automatically	Enable zone redundancy
Seconds of failover, lowest latency, readable replica	Business Critical (zone-redundant if needed)
Survive a region outage with minutes of RTO and seconds of RPO	Auto-failover group to the paired or another region
Survive a region outage, can tolerate hours of downtime and up to ~1 h data loss	Geo-restore (GRS/GZRS backups) - lowest cost
On-premises SQL Server needs a cloud DR target	Managed Instance link (or SQL Server on Azure VM with Always On)
Rarely used, non-critical dev/test	Built-in HA + PITR is usually enough

> **Remember**
> HA and DR protect against *infrastructure* failures. They do **not** protect against bad data or accidental deletes - a deleted row is replicated instantly. That is what backups, PITR and LTR are for. You need **both**.

9. Migration to Azure SQL

Migrating a database is less about copying data and more about **reducing surprises**. A good plan assesses compatibility early, picks the right target and method, rehearses the cutover and validates carefully afterwards.

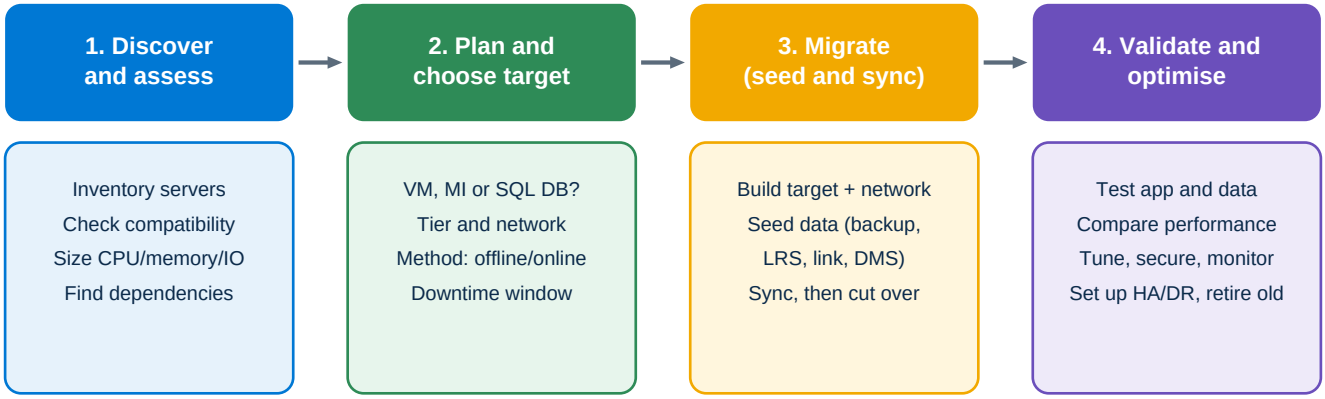


Figure 9.1 - The four phases of a database migration

9.1 Phase 1: Discover and assess

Start by finding out *what you have* and *what will break*. Useful tools and approaches:

- **Azure Migrate** - discovers SQL Server instances, assesses readiness for each Azure SQL target and recommends a right-sized SKU based on performance data.
- **Azure Arc-enabled SQL Server** - gives inventory, best-practice and migration assessment from the Azure portal, and can start a migration to Managed Instance from there.
- **SQL Server Migration Assistant (SSMA)** - for non-SQL Server sources (Oracle, MySQL, Db2, SAP ASE, Access): converts schema and moves data.
- **Azure Data Studio's Azure SQL Migration extension and Data Migration Assistant** - widely used in the past. Microsoft is retiring or replacing several older tools, so check the Azure Database Migration Guide for the currently recommended tooling before you start.

Assess these things for every database:

Question	Why it matters
Which features are used? (Agent, linked servers, CLR, FILESTREAM, cross-DB queries, Service Broker, DTC)	Decides whether SQL Database is possible or MI/VM is needed
How big is it, and how fast does it grow?	Sets tier, storage limits (Hyperscale vs MI), and copy time
What are CPU, memory, IO and peak patterns?	Right-sizing, which controls cost and performance
How much downtime is acceptable?	Offline vs online migration method
What connects to it? (apps, ETL, reports, jobs)	Connection strings, firewall/network, authentication changes
Licensing? (Software Assurance, Hybrid Benefit)	Cost, reservations
Which SQL Server version and compatibility level?	Upgrade path and testing effort

9.2 Phase 2: Choose the target and the method

Choose...	When
SQL Database (single, pool or Hyperscale)	App can work at database scope; modernising; need serverless, elastic pools or Hyperscale; no heavy dependence on instance features
Managed Instance	Need SQL Agent, cross-database queries, linked servers, CLR or other instance features; want minimal changes; need native restore or MI link
SQL Server on Azure VM	Need OS access, an unsupported version/feature, or third-party software on the same host

Then choose a migration method. The main trade-off is **downtime vs effort**:

Method	Target	Type	Downtime	Best for
Native backup and RESTORE FROM URL	MI	Offline	Backup + copy + restore time	Small and medium DBs, simplest approach
Log Replay Service (LRS)	MI	Online (log shipping)	Short (final log + cutover)	Large DBs; you keep taking log backups to Blob Storage
Managed Instance link	MI	Online (distributed AG)	Minutes	Large or critical DBs; fall-back and read offload options
Azure Database Migration Service (DMS)	SQL DB, MI, VM	Offline or online (depends on target)	Varies	Guided, portal-driven migrations
Transactional replication	SQL DB, MI	Online	Low	Selected tables, complex scenarios
BACPAC (export / import)	SQL DB, MI	Offline	Long (grows with size)	Small DBs, portability, cross-version moves
Azure Data Factory / bcp / SSIS	Any	Offline or incremental	Varies	Table copies, transformations, partial migrations

9.3 Phase 3: Migrating to Managed Instance

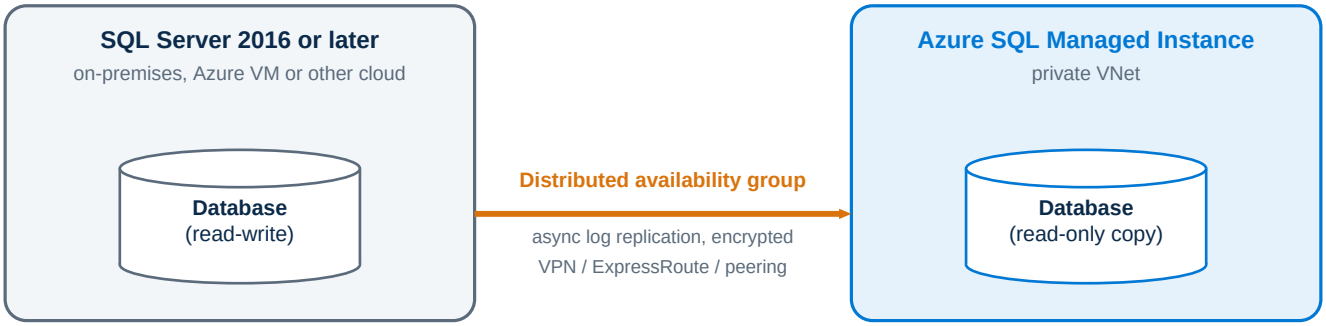
Option A: Native backup and Log Replay Service (LRS)

A simple, reliable pattern when you can reach Azure Storage from the source: copy full, differential and log backups to Blob Storage and have MI restore them. With LRS, MI keeps restoring new log backups until you decide to cut over, so downtime is short.

- **1. Prepare** - create the MI (right-sized), the network path, a Storage account and container. Plan authentication (SAS token or managed identity).
- **2. Back up** - take a full backup *with CHECKSUM* of each database to Blob Storage (one folder per database).
- **3. Start LRS** - start the restore job on MI (PowerShell, CLI or portal), pointing at the container.
- **4. Keep it current** - continue to upload differential and log backups; MI applies them as they arrive.
- **5. Cut over** - stop the application, take the final (tail) log backup, upload it, complete LRS so the database comes online.
- **6. Repoint and validate** - change connection strings to MI, run smoke tests and watch performance.

Option B: Managed Instance link (near-zero downtime)

The MI link uses SQL Server's **distributed availability group** technology to continuously replicate a database from SQL Server (2016 or later; support details depend on the version and update level) to MI over an encrypted connection. The MI copy is readable while it is being synchronised, so you can test with real data before cutting over.



Cutover: stop writes, let the link catch up, fail over. The MI copy becomes read-write. Downtime: minutes.

Figure 9.2 - Managed Instance link

The link is useful for three things: (1) **online migration** with minutes of downtime, (2) **disaster recovery** from on-premises to Azure, and (3) **offloading read-only or reporting workloads** to MI. Newer SQL Server versions also support failing back from MI to SQL Server, which gives you a rollback path. Network requirements include connectivity between the two sides and open endpoint ports (such as 5022 and MI's redirect port range), so involve your network team early.

9.4 Migrating to Azure SQL Database

- **Schema first** - extract and deploy the schema with a DACPAC (SqlPackage / SQL projects), fix unsupported features, then load data.
- **Data move** - DMS (offline), BACPAC import for small databases, Azure Data Factory or bcp for table copies, and **transactional replication** for low-downtime cutover.
- **Replace what is missing** - SQL Agent jobs become **Elastic Jobs**, Azure Automation, Logic Apps or Azure Functions; cross-database queries become elastic query or a redesign; linked servers are replaced by app-level integration or Data Factory.
- **Identity** - convert server logins to **contained database users** (ideally Entra identities).
- **Large databases** - consider Hyperscale (up to 128 TB) or Managed Instance if you exceed single-database size limits.

9.5 Phase 4: Validate and optimise

Step	Details
Functional validation	Row counts and checksums, key reports, batch jobs, integrations, security tests
Compatibility level	Keep the original level first to avoid regressions; use Query Store to compare, then raise it deliberately
Statistics and indexes	Update statistics, rebuild fragmented indexes (index and stats maintenance is still your job)
Performance baseline	Capture Query Store, CPU, IO before and after; right-size the tier (up or down)
Jobs and alerts	Recreate SQL Agent jobs (MI) or their replacements (SQL DB); configure Azure Monitor alerts
Security	Enable Entra auth, Defender for SQL, auditing, private endpoint, TDE settings
Backup and DR	Set PITR/LTR retention and backup redundancy; configure zone redundancy and failover groups; test restores
Cost	Apply Hybrid Benefit and reservations; remove unused resources; schedule dev/test stop/start
Decommission	Keep the source read-only for an agreed period, then retire it

9.6 Common pitfalls and how to avoid them

Pitfall	Prevention
Not enough network planning for MI (subnet too small, DNS not forwarded, overlapping IPs)	Design the subnet, NSG, routes and DNS first; test connectivity from on-premises early
Different collation or time zone than the source (instance-level on MI)	Set collation and time zone when creating the MI; they cannot easily be changed later
Cutover surprises	Rehearse the whole cutover at least once with a production-size copy; write a time-boxed runbook with a rollback decision point
App breaks on login (SQL logins, hard-coded server names, old TLS)	Update connection strings, drivers and authentication; test every application and job
Performance differs from on-premises	Compare with Query Store; right-size; test storage-heavy workloads on the right tier (for example BC or Next-gen GP)
Forgetting that PaaS has quotas and limits	Check resource limits (databases per instance, storage, vCores, connections) against your estate
Underestimating transfer time	Estimate bandwidth (for example ExpressRoute or Azure Data Box for very large data), and use online methods for big databases

9.7 Sample cutover runbook (online migration)

When	Action	Owner
T - 1 week	Final rehearsal complete; go / no-go review; communicate window	Project lead
T - 1 day	Verify replication/log shipping is healthy and lag is small	DBA
T - 0:30	Freeze deployments; confirm backups; notify stakeholders	DBA / App team
T - 0:15	Stop application writes (maintenance mode)	App team
T	Final log backup / wait until link is synchronised; complete cutover	DBA
T + 0:15	Update connection strings / DNS; start application	App team
T + 0:30	Smoke tests, monitoring, check error logs	Everyone
T + 2:00	Go / no-go decision on keeping Azure as primary or rolling back	Project lead
T + 1 week	Performance review, cost check, decommission plan for the source	DBA / Finance

Appendix A: One-page Cheat Sheet

I need to...	Use...
Run a new cloud app with a managed database	Azure SQL Database (serverless for spiky use, Hyperscale for very large)
Move an existing SQL Server with minimal changes	Managed Instance (native restore, LRS, or MI link)
Run many small tenant databases cheaply	SQL Database elastic pool
Keep the database private	Private endpoint (SQL DB); VNet-native endpoint (MI); disable public access
Use passwordless access from apps	Managed identity + Entra authentication
Encrypt data at rest with my own key	TDE with customer-managed key in Key Vault
Hide sensitive column values from some users	Dynamic Data Masking (and column permissions)
Keep secrets from even the DBA	Always Encrypted
Undo an accidental delete or bad deployment	Point-in-time restore to a new database
Keep yearly copies for 7 years	Long-term retention policy
Survive a zone failure	Zone redundancy
Survive a region failure	Auto-failover group (or geo-restore for low cost)
Move a huge database with minutes of downtime	Managed Instance link (MI) or transactional replication
Find the slow query	Query Store / Query Performance Insight
Get alerted on high CPU or low storage	Azure Monitor metric alerts + action groups
Cut licence cost	Azure Hybrid Benefit + reservations; stop dev/test instances

Appendix B: Glossary

Term	Meaning
ARM	Azure Resource Manager - the deployment and management layer of Azure
AG / DAG	Availability Group / Distributed Availability Group - SQL Server replication technology behind HA and MI link
AZ	Availability Zone - a separate data centre inside a region
BACPAC	A portable file containing a database's schema and data
BC / GP	Business Critical / General Purpose service tiers
BYOK / CMK	Bring your own key / customer-managed key for encryption
DTU	Database Transaction Unit - bundled performance unit in the older purchasing model
Entra ID	Microsoft's identity service (formerly Azure Active Directory)
GRS / ZRS / LRS	Geo-/Zone-/Locally redundant storage
LRS (migration)	Log Replay Service - restores log-shipped backups into MI for online migration
LTR / PITR	Long-term retention / Point-in-time restore
MI	Azure SQL Managed Instance

Term	Meaning
NSG	Network Security Group - network firewall rules attached to subnets or NICs
Private Endpoint	A private IP in your VNet that maps to a PaaS service via Private Link
Query Store	Built-in history of queries, plans and runtime statistics
RPO / RTO	Recovery Point / Time Objective
TDE	Transparent Data Encryption - encryption of data at rest
vCore	Virtual core - the compute unit of the vCore purchasing model
VNet	Virtual Network - your private network in Azure

Appendix C: Where to Learn More

Azure SQL evolves quickly: limits, tiers and tools change several times a year. Treat this document as an architecture guide and confirm exact numbers on Microsoft Learn before final design.

Topic	Where to look on Microsoft Learn (learn.microsoft.com)
Everything Azure SQL	Azure SQL documentation (azure-sql)
Sizing and limits	'Resource limits' pages for SQL Database and Managed Instance
Choosing between options	'Azure SQL' product overview and feature comparison articles
Business continuity	'Business continuity overview' for SQL Database and Managed Instance
Migration tooling	Azure Database Migration Guide
Pricing and SLAs	Azure pricing calculator and the 'SLA for Azure SQL' pages